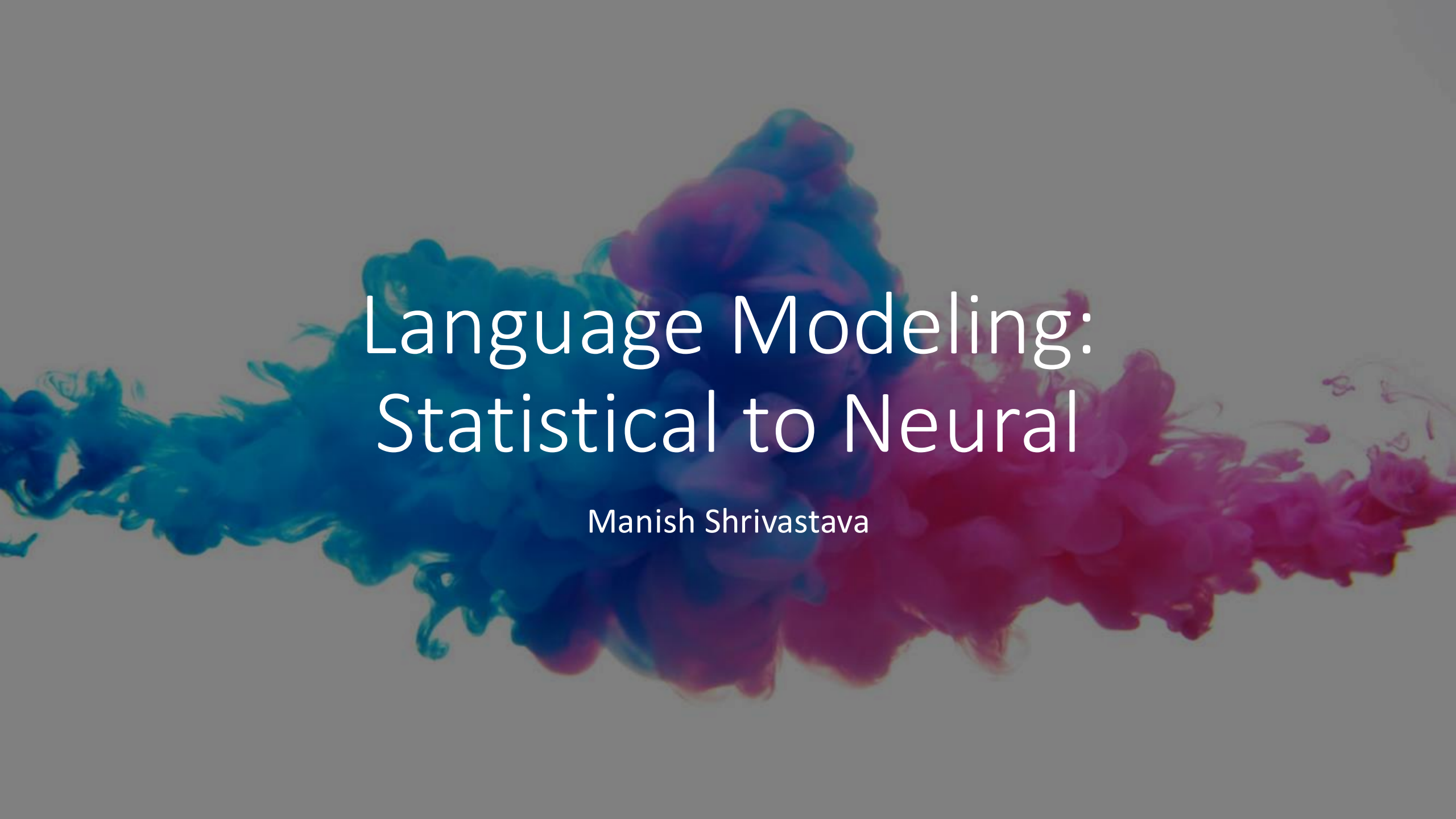




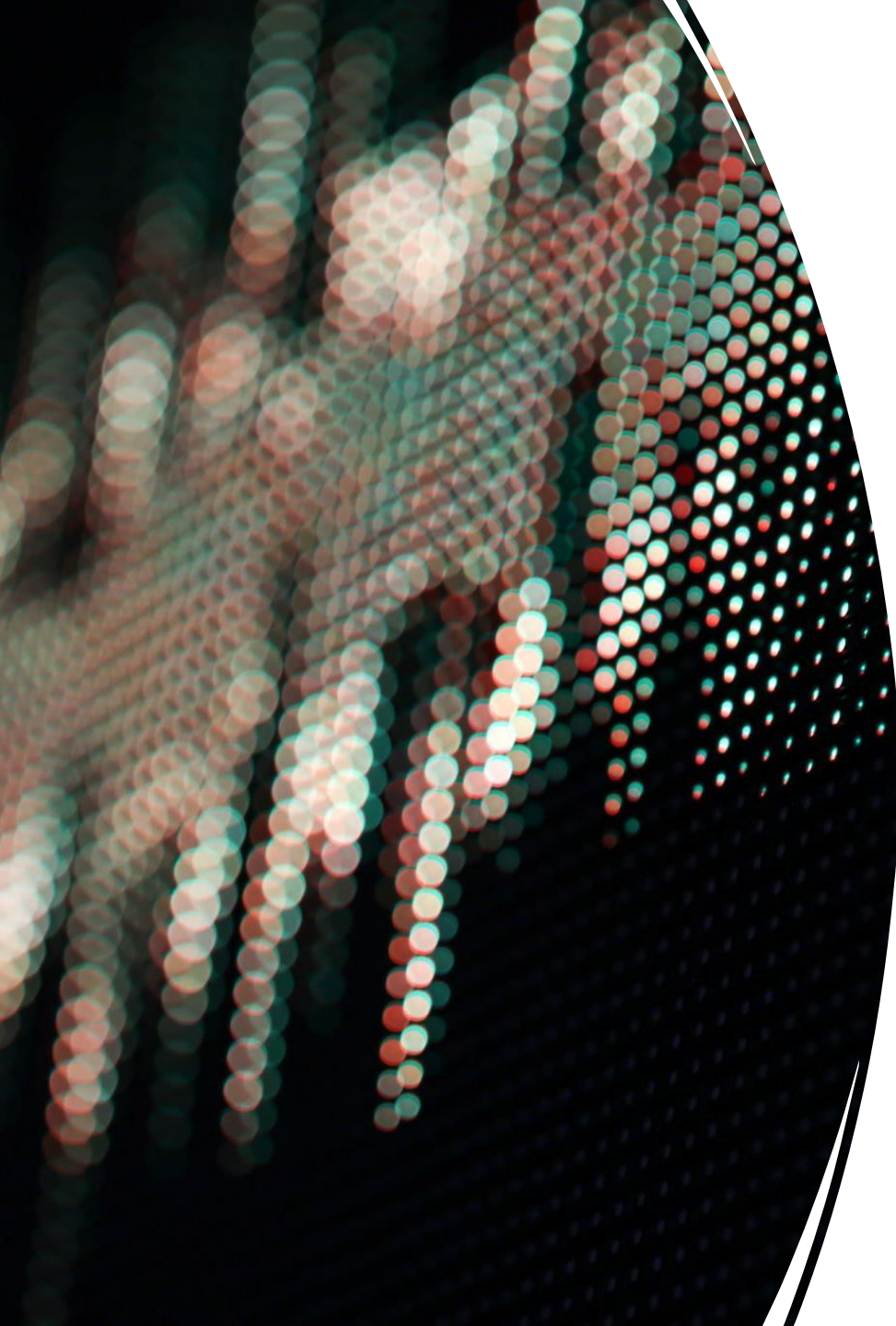
Language Modeling: Statistical to Neural

Manish Shrivastava



Outline

- Language Modeling
 - Modeling
 - Statistical
 - N-grams
 - Challenges
 - Sparsity
 - Meaning
- Neural Representation (why?)
- Neural Language Modeling
 - RNN



Statistical model

- Built of
 - “statistics”
 - Features
- Approximation of “population” or “process” or “phenomenon”
- The phenomenon we are interested in
 - Language

Statistical Estimators

- Given the observed training data ...
 - How do you develop a model (probability distribution) to predict future events?
 - Language Modeling
 - Predict Likelihood of sequences

Language Modelling Problem

- Aim is to calculate the probability of a sequence (sentence) $P(X)$
- Can be decomposed into product of conditional probabilities of tokens (works):

- $$P(W) = P(w(1), w(2), w(3), \dots, w(M+1)) = \prod_{k=1}^{M+1} P(w(k) | w(1), \dots, w(k-1))$$

$$P(w(k) | w(1), \dots, w(k-1)) \approx P(w(k) | w(1), \dots, w(k-N+1))$$

Basic Idea:

- Examine short sequences of words
- How likely is each sequence?
- “Markov Assumption” – word is affected only by its “prior local context” (last few words)



Example

- The boy ate a chocolate
 - The girl bought a chocolate
 - The girl then ate a chocolate
 - The boy bought a horse
-
- Can we figure out how likely is the following sentence
 - The boy bought a chocolate

“Shannon Game”

- Claude E. Shannon. “Prediction and Entropy of Printed English”, *Bell System Technical Journal* 30:50-64. 1951.
- Predict the next word, given $(n-1)$ previous words
- Determine probability of different sequences by examining training corpus

Forming Equivalence Classes (Bins)

- “*n-gram*” = sequence of *n* words

- bigram
- trigram
- four-gram or quadrigram

- Probabilities of *n*-grams

- Unigram
$$p(w) = \frac{c(w)}{N}$$

- Bigram
$$P(w_i | w_{i-1}) = \frac{c(w_i, w_{i-1})}{c(w_{i-1})}$$

- Trigram
$$P(w_i | w_{i-1}, w_{i-2}) = \frac{c(w_i, w_{i-1}, w_{i-2})}{c(w_{i-1}, w_{i-2})}$$

Maximum Likelihood Estimation

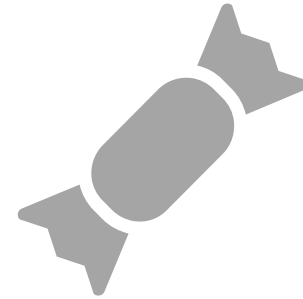
- Data
 - The boy ate a chocolate
 - The girl bought a chocolate
 - The girl then ate a chocolate
 - The boy bought a horse
- The boy bought a chocolate
 - Unigram Probabilities
 - $(4/21) * (2/21) * (2/21) * (4/21) * (3/21)$
 - $(4 * 2 * 2 * 4 * 3) / 21^5 = 0.000047$
 - Bi-gram Probabilities
 - <The boy> <boy bought>
<bought a> <a chocolate>
 - $(2/4) * (2/4) * (2/2) * (3/4) = 0.1875$

Reliability vs. Discrimination



“large green _____”

tree? mountain? frog? car?



“swallowed the large green _____”

pill? candy?

Reliability vs. Discrimination

**larger n: more
information about the
context of the specific
instance (greater
discrimination)**

**smaller n: more
instances in training
data, better statistical
estimates (more
reliability)**

Selecting an n

Vocabulary (V) =
20,000 words

n	Number of bins
2 (bigrams)	400,000,000
3 (trigrams)	8,000,000,000,000
4 (4-grams)	1.6×10^{17}

Maximum Likelihood Estimation

- ▶ $P_{MLE}(w_n | w_1 \dots w_{n-1}) = \frac{C(w_1 \dots w_n)}{C(w_1 \dots w_{n-1})}$
- ▶ Estimate sequence probabilities using “counts” or frequencies of sequences
- ▶ Problems
 - ▶ Sparseness
 - ▶ What do you do when unknown words are seen??

Example

- Data
 - The boy ate a chocolate
 - The girl bought a chocolate
 - The girl then ate a chocolate
 - The horse bought a boy
- The boy bought a chocolate
 - Unigram Probabilities
 - $(4/16)*(2/16)*(2/16)*(4/16)*(3/16)$
 - $(4*2*2*4*3)/21^5 = 0.000047$
 - Bi-gram Probabilities
 - <The boy> <boy bought> <bought a> <a chocolate>
 - $(2/4)*(0/4)*(2/2)*(3/4) = \underline{0}$

Approximating Shakespeare

- Generating sentences with random unigrams...
 - Every enter now severally so, let
 - Hill he late speaks; or! a more to leg less first you enter
- With bigrams...
 - What means, sir. I confess she? then all sorts, he is trim, captain.
 - Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry.
- Trigrams
 - Sweet prince, Falstaff shall die.
 - This shall forbid it should be branded, if renown made it empty.

More-

- Quadrigrams
 - What! I will go seek the traitor Gloucester.
 - Will you not tell me who I am?
- What's coming out here looks like Shakespeare because it *is* Shakespeare
- Note: ***As we increase the value of N , the accuracy of an n -gram model increases, since choice of next word becomes increasingly constrained***

N-Gram Training Sensitivity

- If we repeated the Shakespeare experiment but trained our n-grams on a Wall Street Journal corpus, what would we get?
- Note: ***This question has major implications for corpus selection or design***

WSJ is *not*
Shakespeare:
Sentences
Generated
from WSJ

unigram: Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

bigram: Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

trigram: They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions



Evaluation and Data Sparsity Questions

- Perplexity and entropy: how do you ***estimate*** how well your language model fits a corpus once you're done?
- Smoothing and Backoff : how do you handle unseen n-grams?

Perplexity and Entropy

$$H(X) = - \sum_{x=1}^{x=n} p(x) \log_2 p(x)$$

- Information theoretic metrics
 - Useful in measuring how well a grammar or language model (LM) models a natural language or a corpus
- **Entropy:**
 - With 2 LMs and a corpus, which LM is the better match for the corpus?
 - How much information is there (in e.g. a grammar or LM) about what the next word will be? More is better!
- For a random variable **X** ranging over e.g. bigrams and a probability function **$p(\mathbf{x})$** , the entropy of X is the expected negative log probability

$$PP(W) = 2^{H(W)}$$

Upper bound for entropy:

- **For binary classification**, the upper bound of Entropy is 1.
- **For multi-class classification with each class having same probability**, the upper bound of Entropy will be:

$$Entropy = -[p_1 * \log_2(p_1) + \dots + p_n * \log_2(p_n)] = -\left[\left(\frac{1}{n} * \log_2 \frac{1}{n}\right) + \left(\frac{1}{n} * \log_2 \frac{1}{n}\right) + \dots\right] = -\log_2 \frac{1}{n} = \log_2 n$$

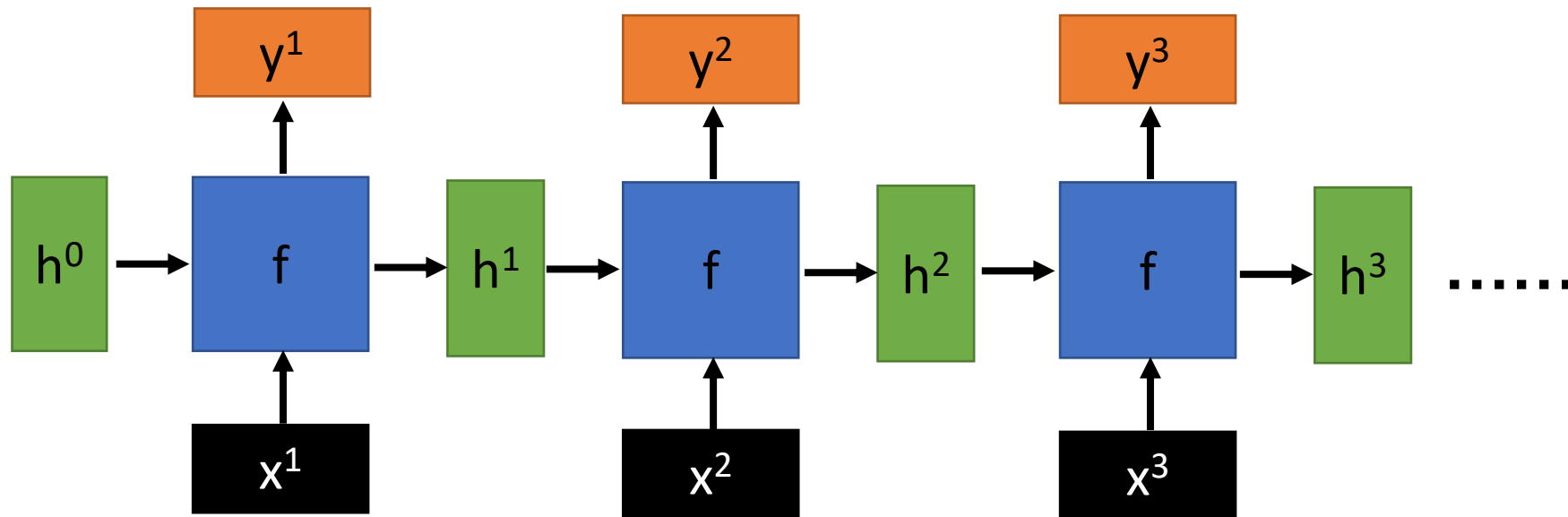
- Entropy is the lower bound on the # of bits it takes to encode information e.g. about bigram likelihood
- **Perplexity**
 - At each choice point in a grammar
 - What are the average number of choices that can be made, weighted by their probabilities of occurrence?
 - I.e., Weighted average branching factor
 - How much probability does a grammar or language model (LM) assign to the sentences of a corpus, compared to another LM? The more information, the lower perplexity

From Julia Hirschberg

How does RNN reduce complexity?

- Given function $f: h', y = f(h, x)$

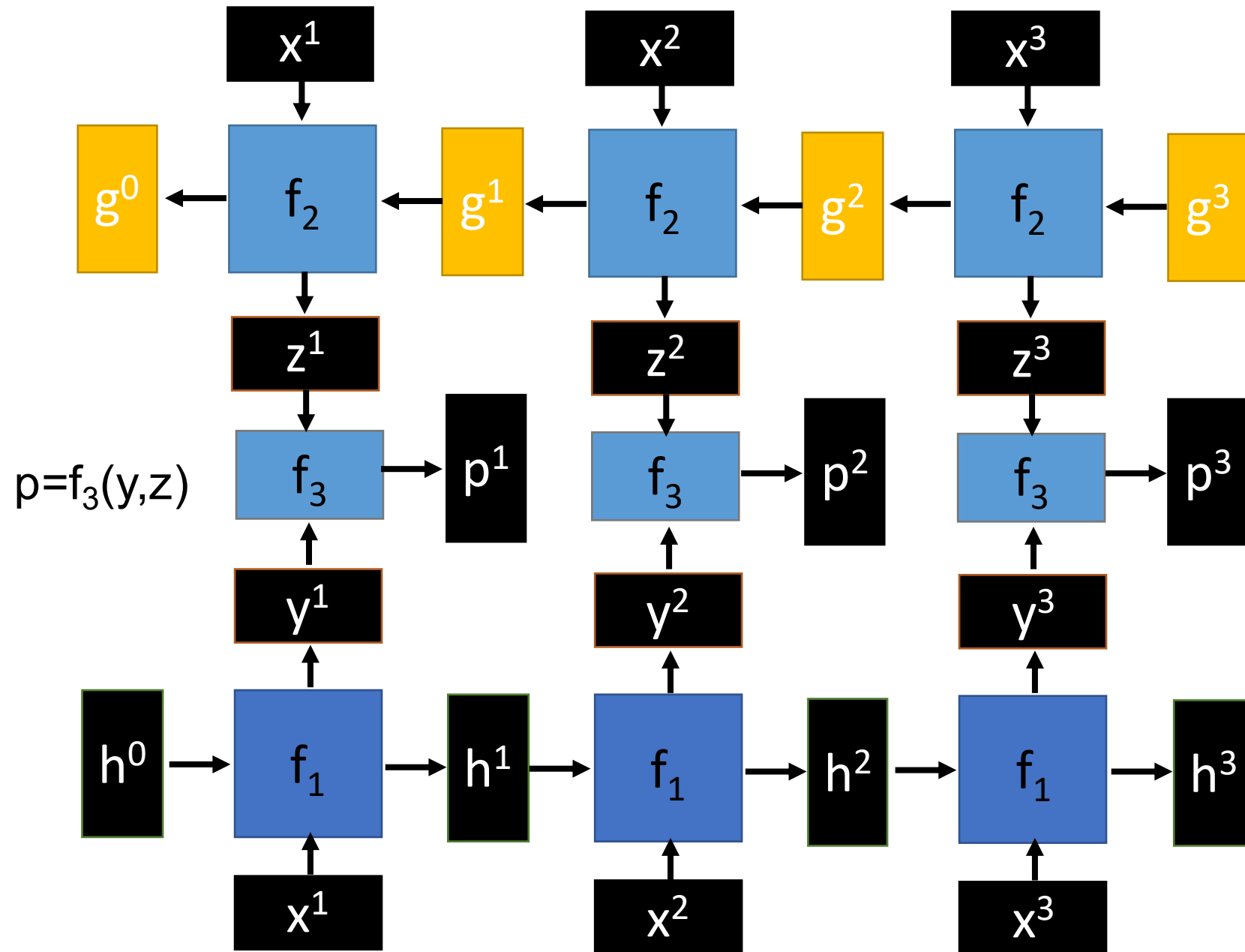
h and h' are vectors with the same dimension



No matter how long the input/output sequence is, we only need one function f . If f 's are different, then it becomes a feedforward NN. This may be treated as another compression from fully connected network.

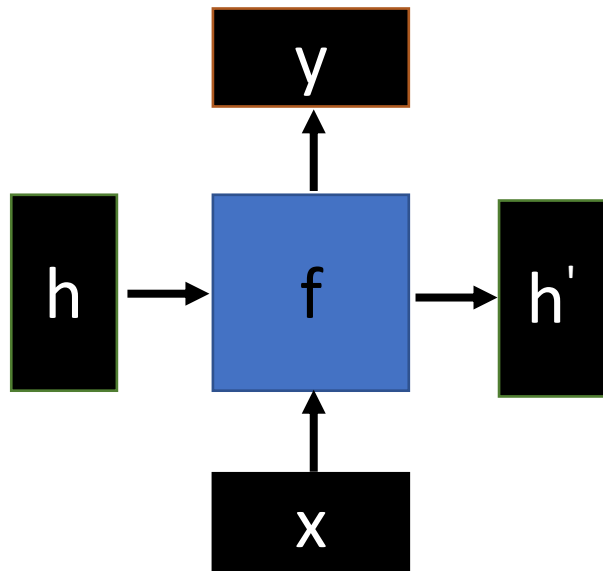
Bidirectional RNN

$$y, h = f_1(x, h) \quad z, g = f_2(g, x)$$



Naïve RNN

- Given function $f: h', y = f(h, x)$



$$h' = \sigma(W^h h + W^i x)$$

$$y = \sigma(W^o h')$$

softmax

Note, y is computed from h'

We have ignored the bias

Problems with naive RNN

- When dealing with a time series, it tends to forget old information. When there is a distant relationship of unknown length, we wish to have a “memory” to it.
- Vanishing gradient problem.

Other Challenges



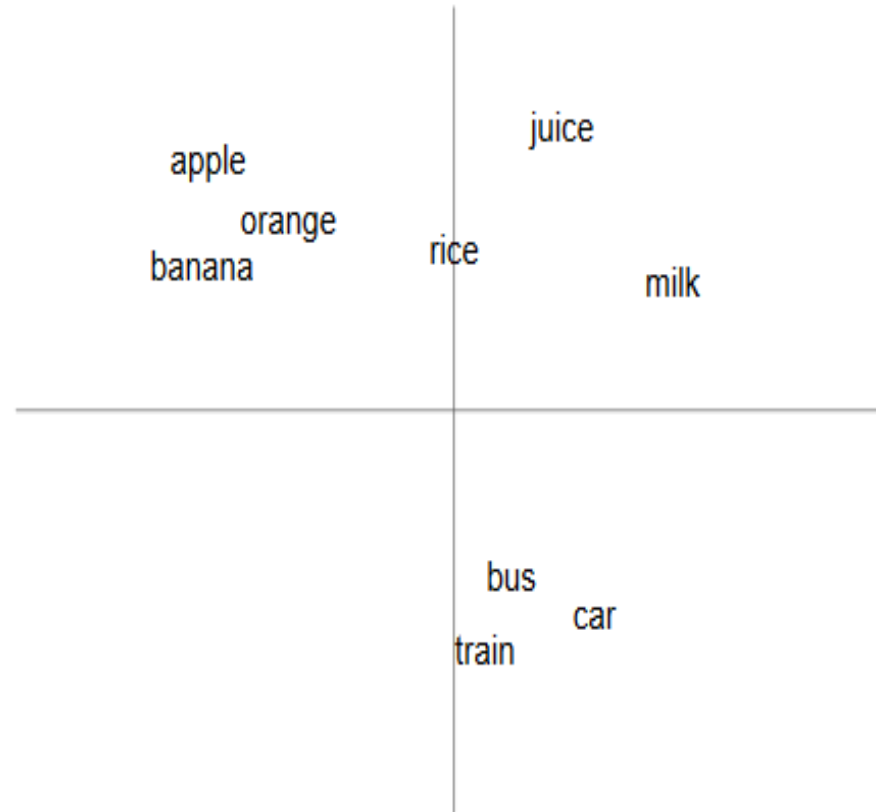
LANGUAGE IS ABOUT
MEANING



WHERE IS MEANING?!

**“You shall know a word by the
company it keeps!”**

Firth (1957)



Distributional Semantics



Key Idea:

The meaning of a word is "distributed" in its context



Takeaway:

We can guess at the meaning by looking at the context



Question:

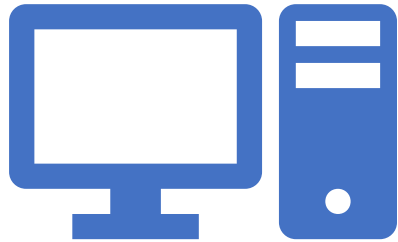
How do we get machines to do it?



Answer:

Start with random "meaning" and keep correcting with the help of the context
till the correct "meaning" is achieved

What is “meaning”



For a computer, its just
numbers!!



We want to represent words,
phrases, sentences as numbers!!

Word Representation

- Aim: Unique numerical representation for each unique word in the dictionary.
- Idea: Consecutive integers
 - Apple = 1
 - Azaleas = 2
 - ...
 - Zebra = 1,09,200
- Seems ok ?

Word Representation

- Aim: Unique and **sensible** numerical representation for each unique word in the dictionary.
- Sensible → All words are 'treated the same'

1-Hot Representation of a Word

- **D1** : Andrew is a tall boy.
- **D2** : Ram is a good boy. Ratna is also good.

Dictionary	A	$[1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0]^T$	} 1-Hot representations
	Also	$[0\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0]^T$	
	Andrew	$[0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0]^T$	
	Boy	$[0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0]^T$	
	Good	$[0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0]^T$	
	Tall	$[0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0]^T$	
	Is	$[0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0]^T$	
	Ram	$[0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0]^T$	
	Ratna	$[0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1]^T$	

Histogram vs. 1-Hot Representation

- Histogram is the sum of one-hot representation of all the words

- **D1:** Andrew is a tall boy

A	0	0	1	0	0	1	0	0	0
Also	0	0	0	0	0	0	0	0	0
Andrew	1	0	0	0	0	0	0	0	0
Boy	0	0	0	0	0	1	0	0	0
Good	0	0	0	0	0	0	0	0	0
Tall	0	0	0	0	1	0	0	0	0
Is	0	1	0	0	0	0	0	0	0
Ram	0	0	0	0	0	0	0	0	0
Ratna	0	0	0	0	0	0	0	0	0

Document Histogram

- **Histogram for both the documents:**

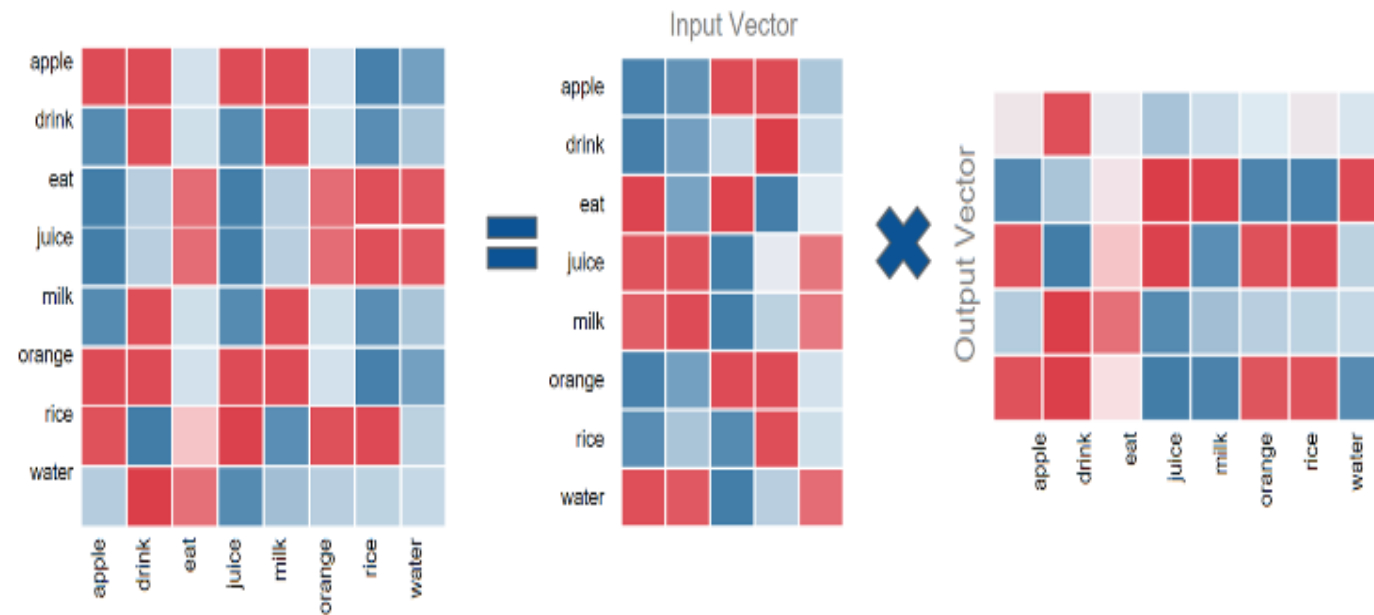
Dictionary	A	1	1
	Also	0	1
	Andrew	1	0
	Boy	1	1
	Good	0	2
	Tall	1	0
	Is	1	2
	Ram	0	1
	Ratna	0	1

- **D1** : Andrew is a tall boy.
- **D2** : Ram is a good boy.
Ratna is also good.

Representing Words

- Words are atomic entities
- One-hot representation does not capture the meaning
 - Words with similar or different meanings are equally apart
 - Cosine distance is 0 between any two words
 - This is unlike apples and oranges
- Can words be vectors with meaningful distances?
 - How do we capture meaning?
 - How do we learn this from unlabeled data?

Singular Value Decomposition



The problem with this method, is that we may end up with matrices having billions of rows and columns, which makes SVD computationally restrictive.

Singular Value Decomposition (SVD)

- Handy mathematical technique that has application to many problems
- Given any $m \times n$ matrix **A**, algorithm to find matrices **U**, **V**, and **W** such that

$$\mathbf{A} = \mathbf{U}\mathbf{W}\mathbf{V}^T$$

U is $m \times n$ and orthonormal

W is $n \times n$ and diagonal

V is $n \times n$ and orthonormal

SVD

$$\begin{pmatrix} \mathbf{A} \end{pmatrix} = \begin{pmatrix} \mathbf{U} \end{pmatrix} \begin{pmatrix} w_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & w_n \end{pmatrix} \begin{pmatrix} \mathbf{V} \end{pmatrix}^T$$

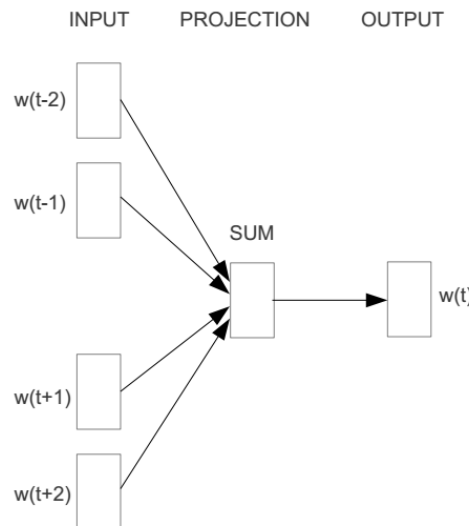
- Treat as black box: code widely available

SVD

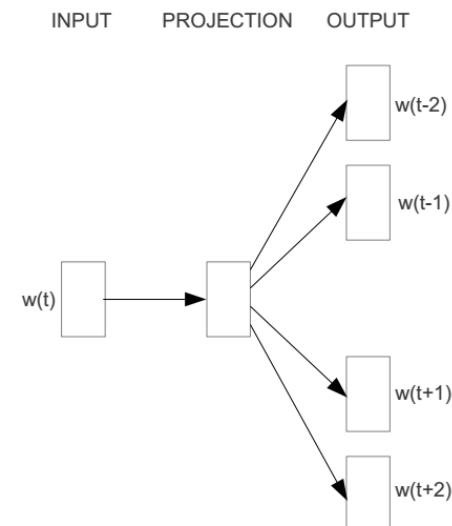
- The w_i are called the **singular values** of **A**
- If **A** is singular, some of the w_i will be 0
- In general $\text{rank}(\mathbf{A}) = \text{number of nonzero } w_i$
- SVD is mostly unique (up to permutation of singular values, or if some w_i are equal)

Represent the meaning of word **Word2vec**

- 2 basic neural network models:
 - Continuous Bag of Word (CBOW): use a window of word to predict the middle word
 - Skip-gram (SG): use a word to predict the surrounding ones in win



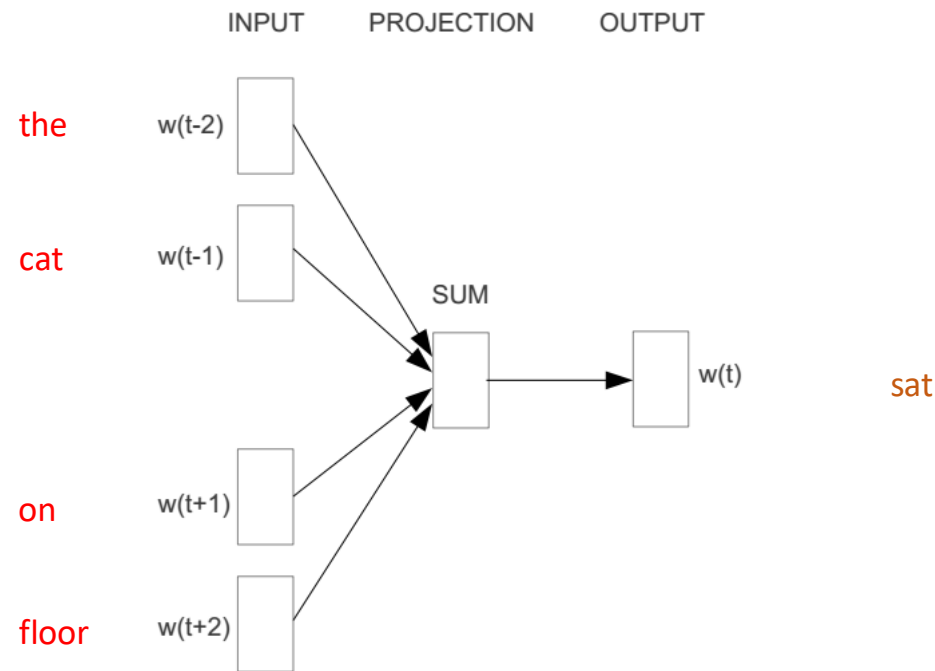
CBOW

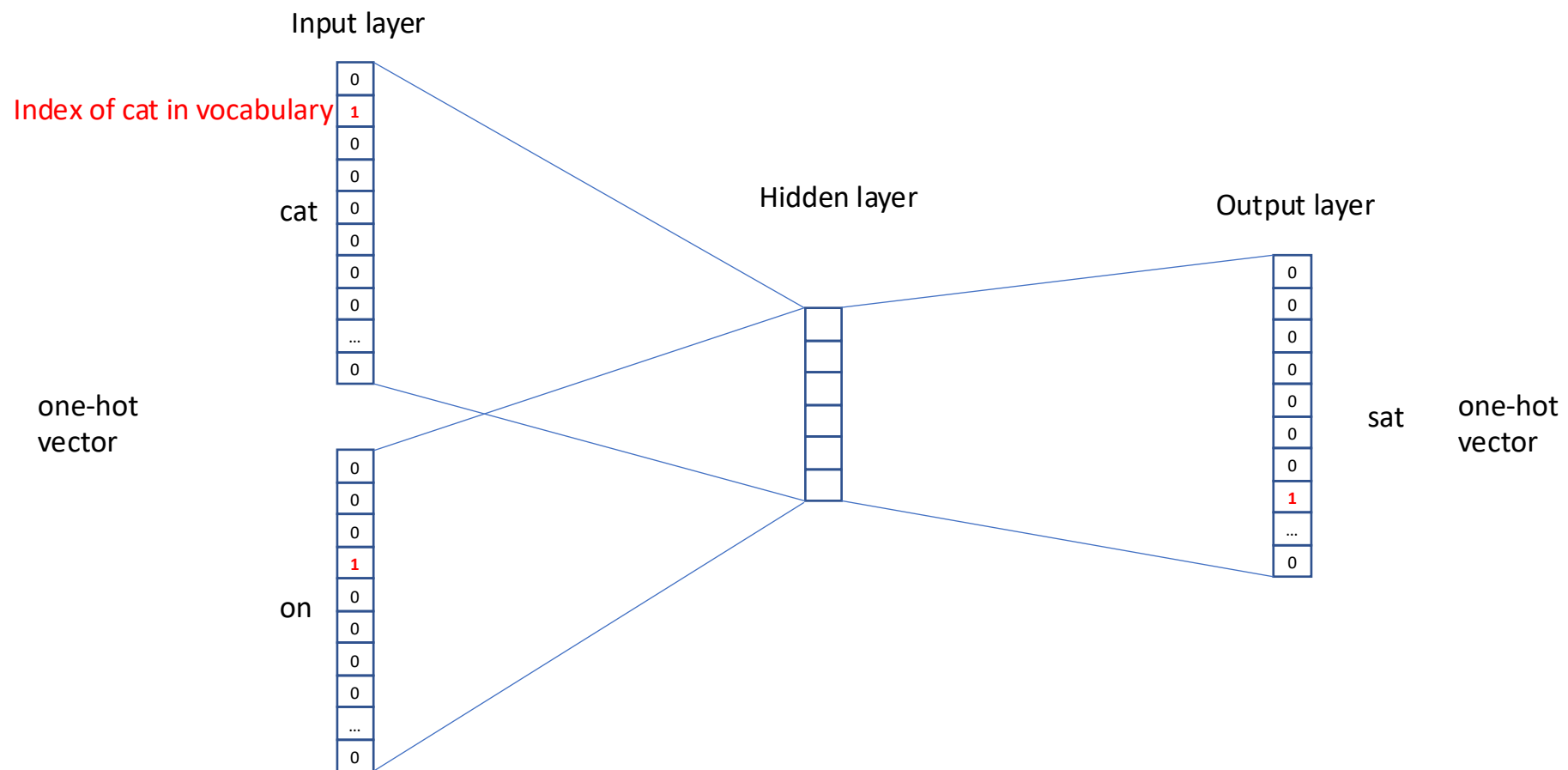


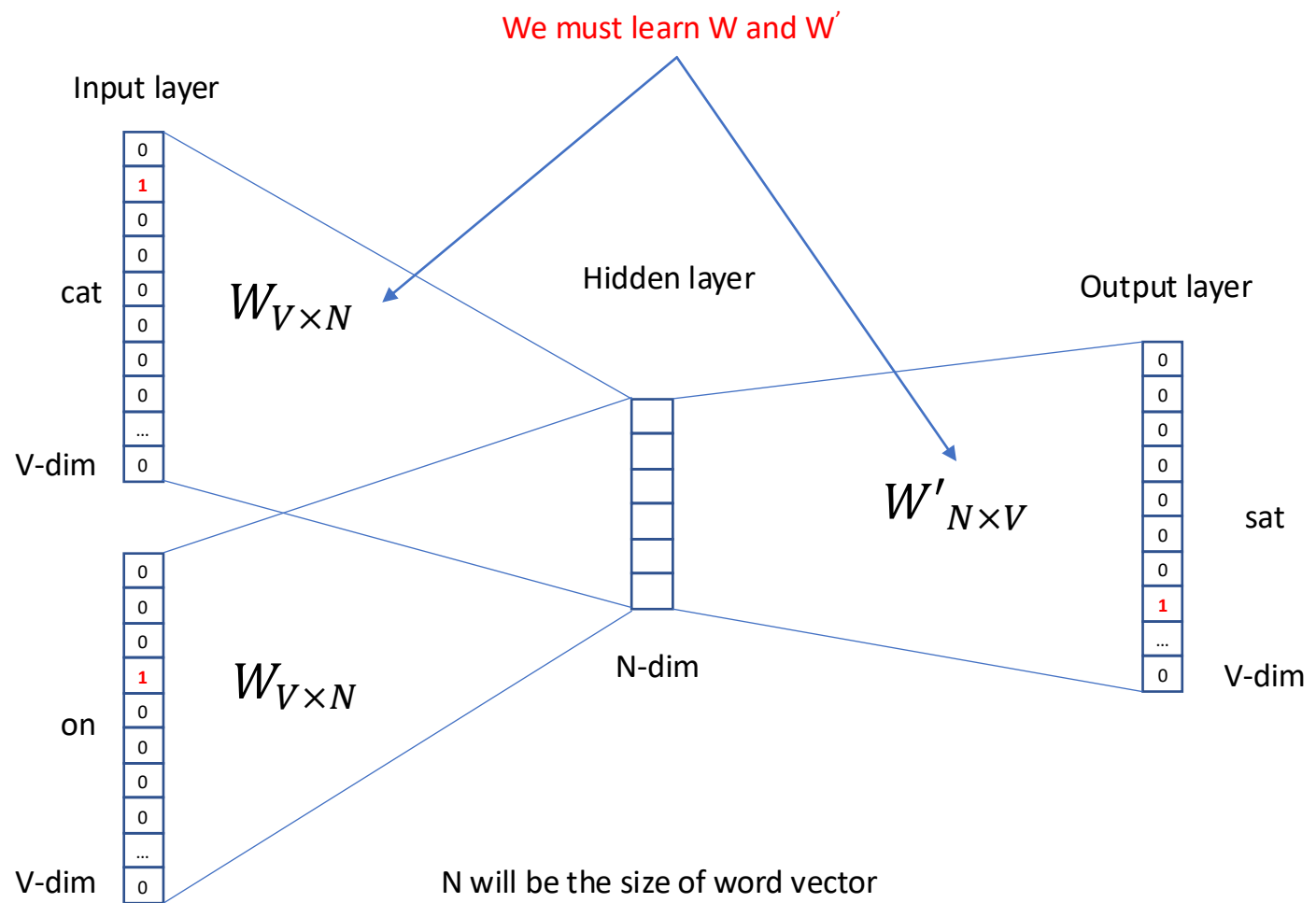
Skip-gram

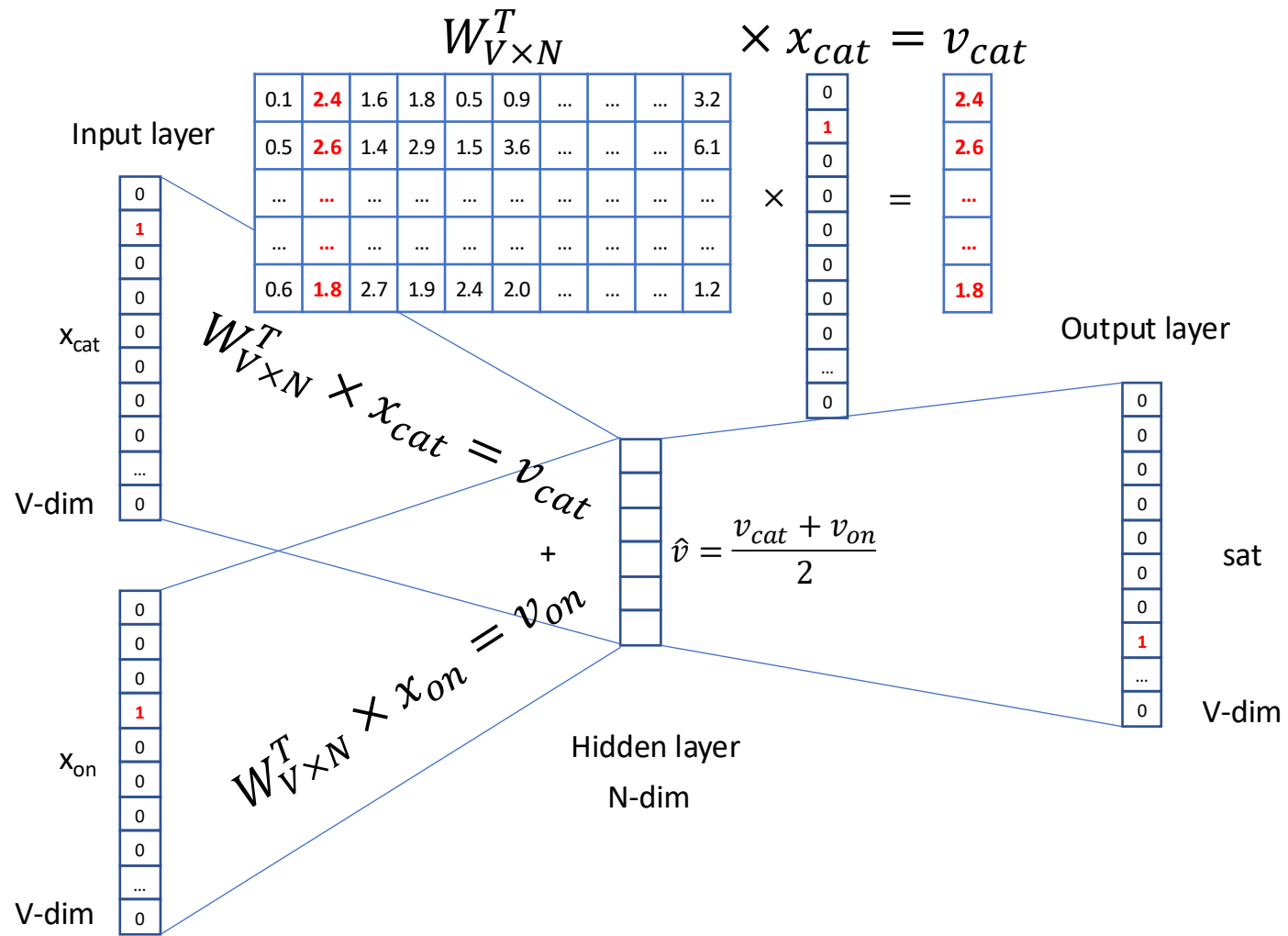
Word2vec – Continuous Bag of Word

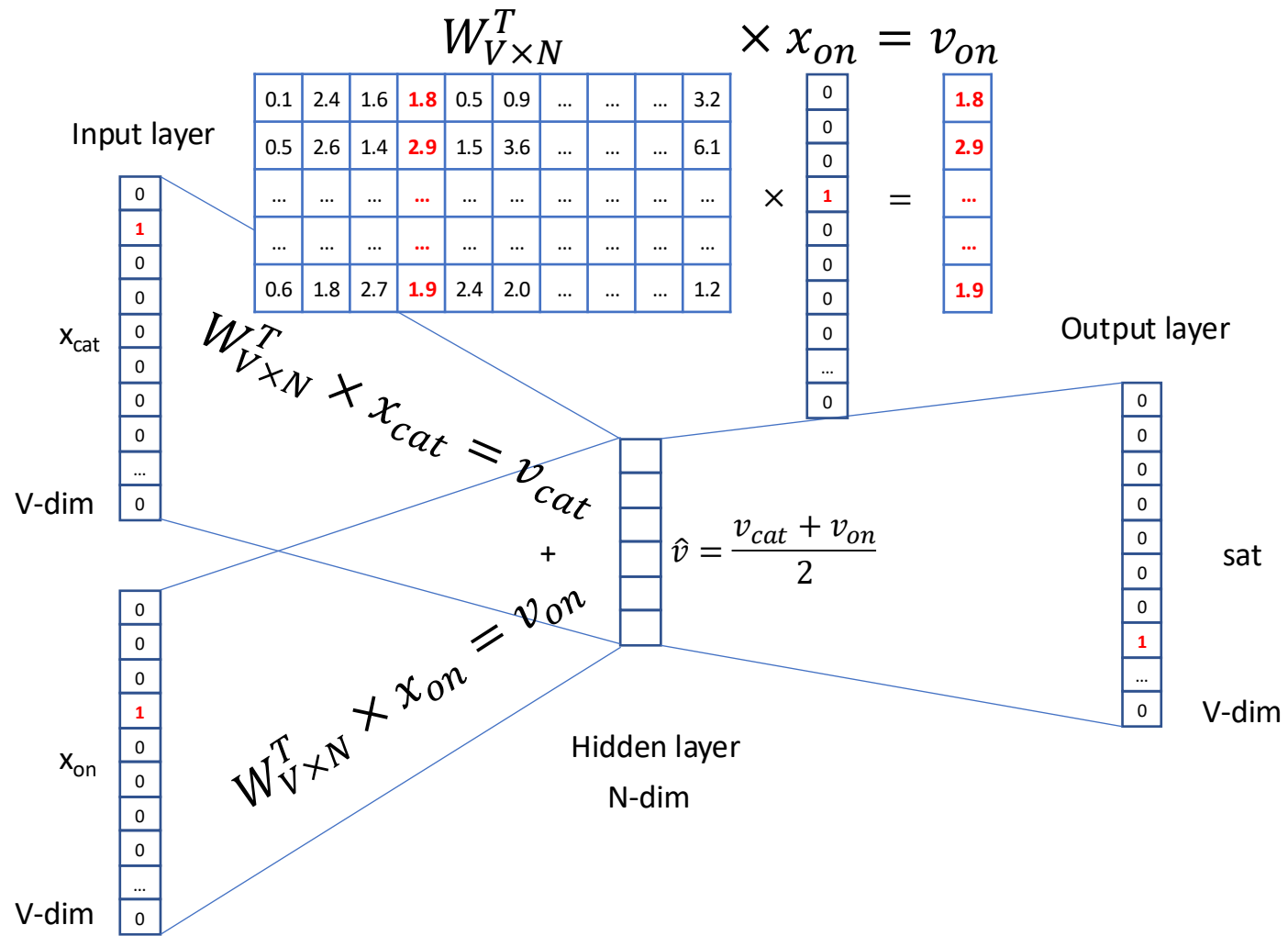
- E.g. “The cat sat on floor”
 - Window size = 2

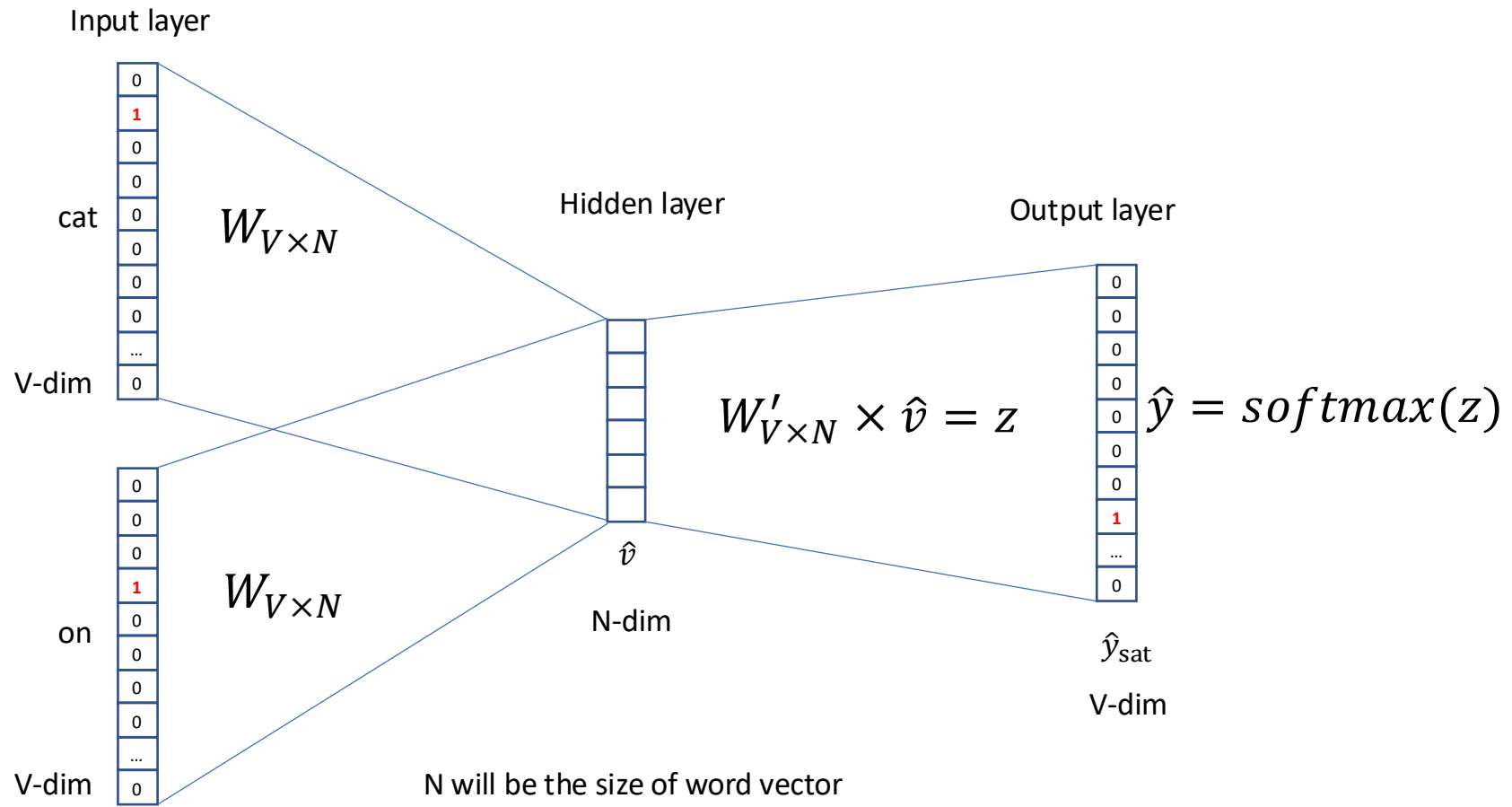


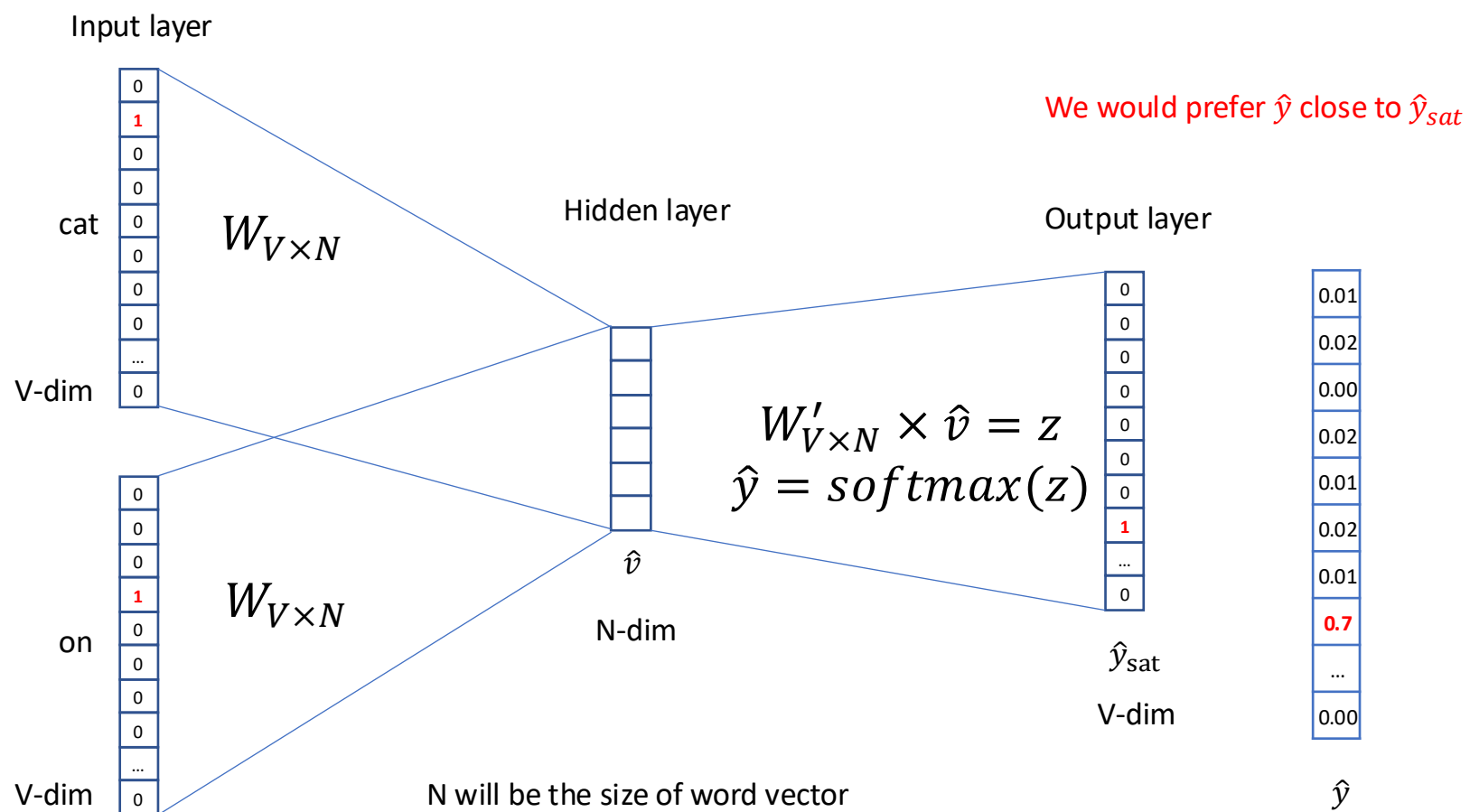


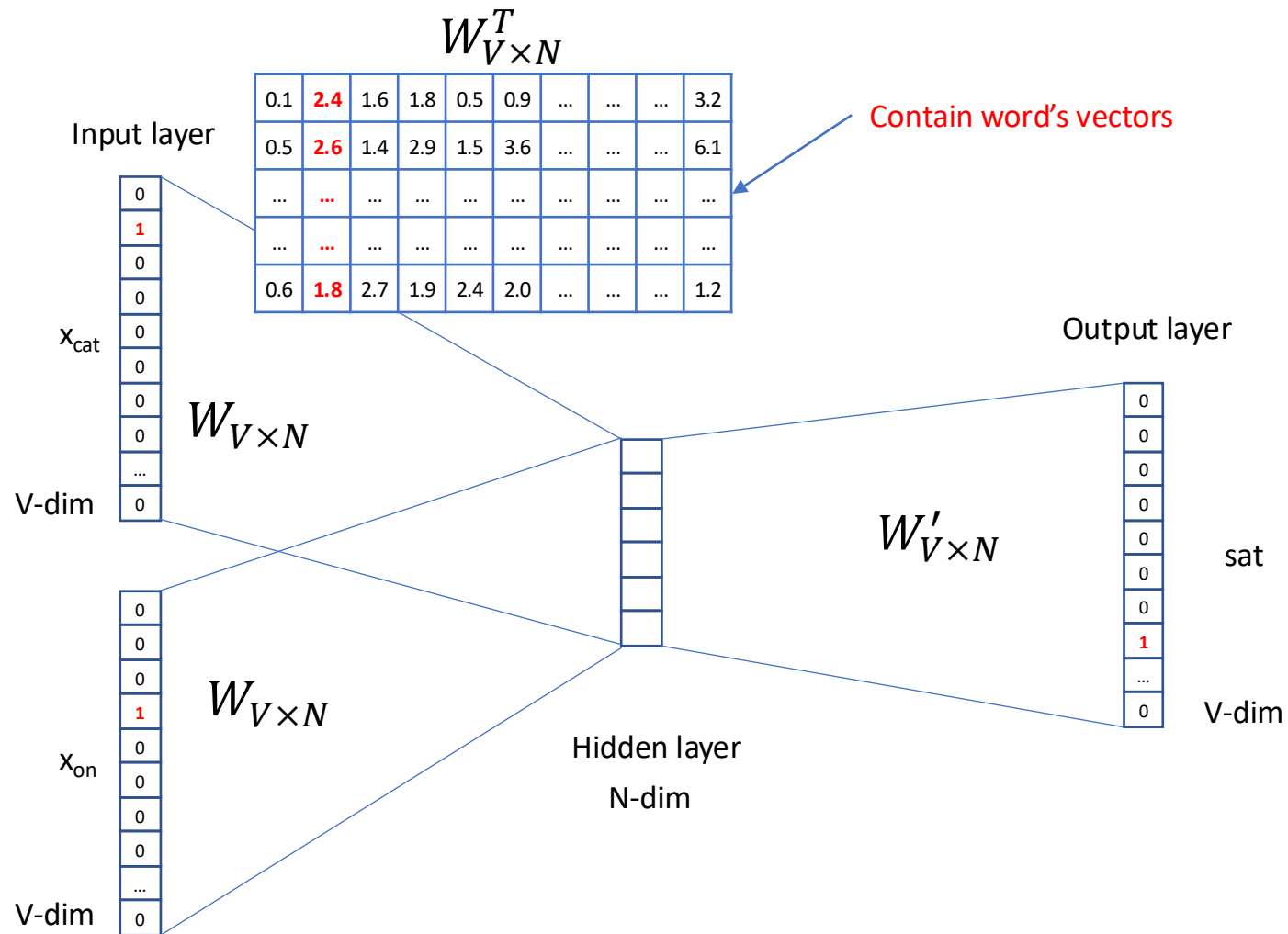












We can consider either W or W' as the word's representation. Or even take the average.

Some interesting results

Word Analogies

Test for linear relationships, examined by Mikolov et al. (2014)

a:b :: c:?



$$d = \arg \max_x \frac{(w_b - w_a + w_c)^T w_x}{||w_b - w_a + w_c||}$$

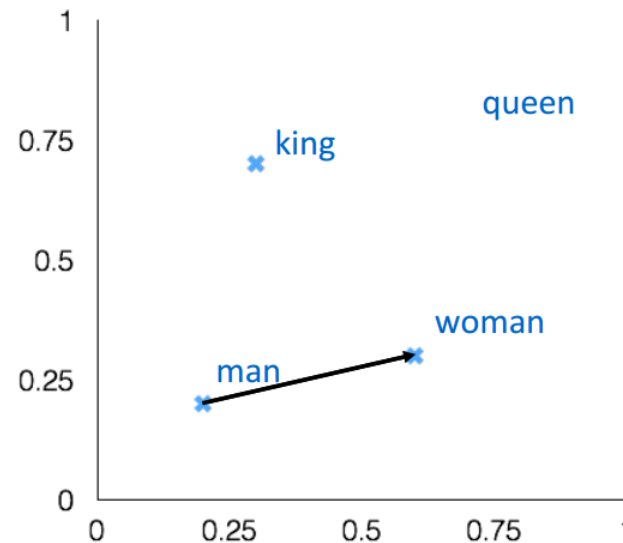
man:woman :: king:?

+ king [0.30 0.70]

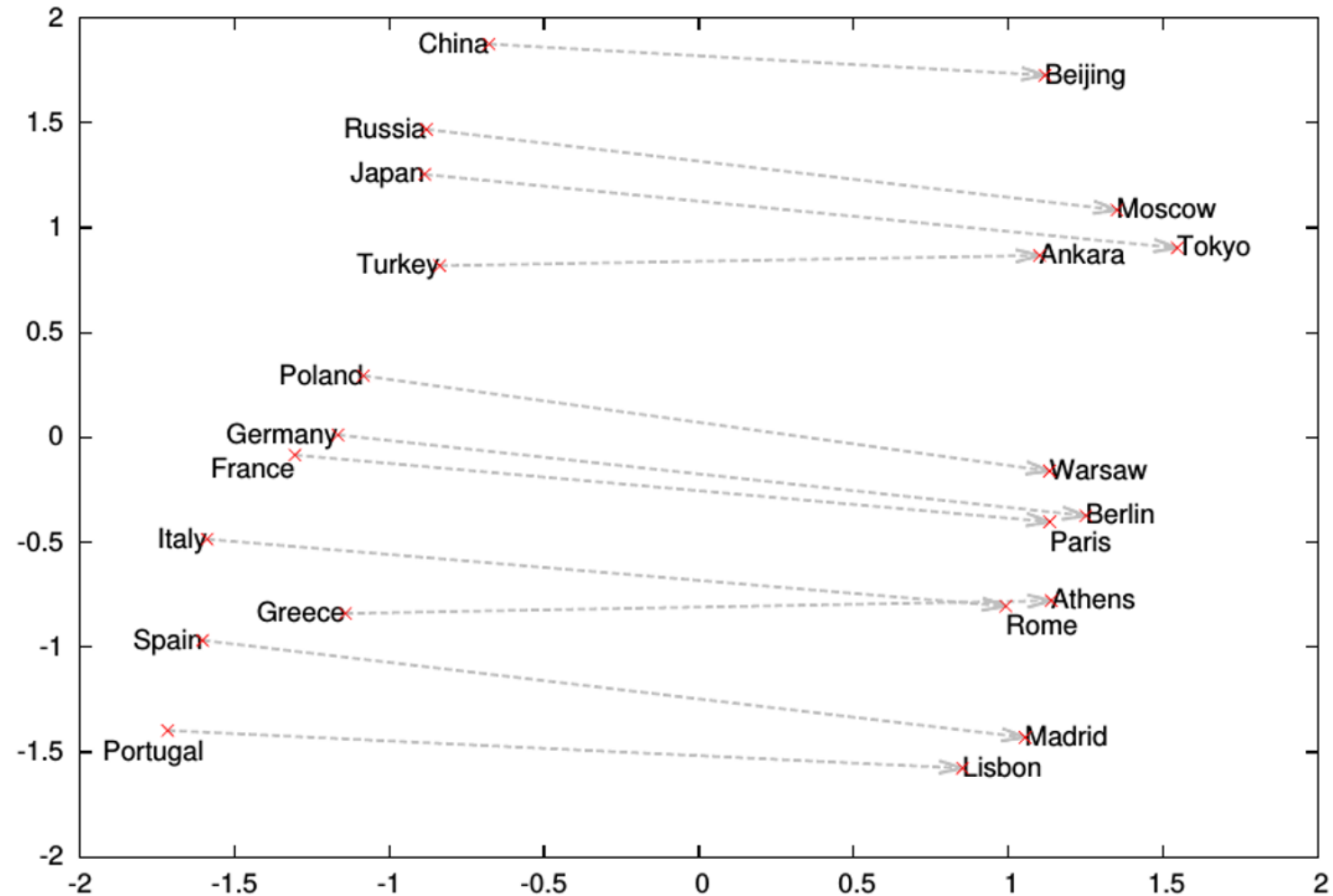
- man [0.20 0.20]

+ woman [0.60 0.30]

queen [0.70 0.80]



Word analogies



Now what?

- Word2Vec == Deep Learning
 - No!
 - The first step
- Embeddings are dense vectors
 - Can encapsulate context information
 - Can serve as information points similar to pixels
 - Serve as input to Deep learning models
 - CNNs, RNNs and many more

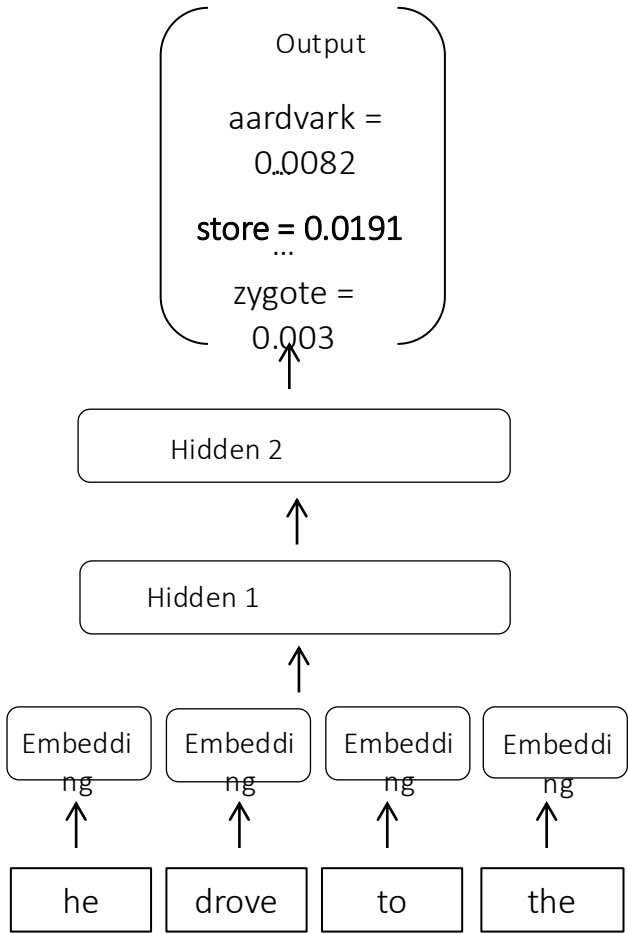
Deep Learning and NLP

- Tasks
 - Language Modeling
 - Semantic encoding of larger sentential units
 - Semantic similarity at larger granularity for various tasks

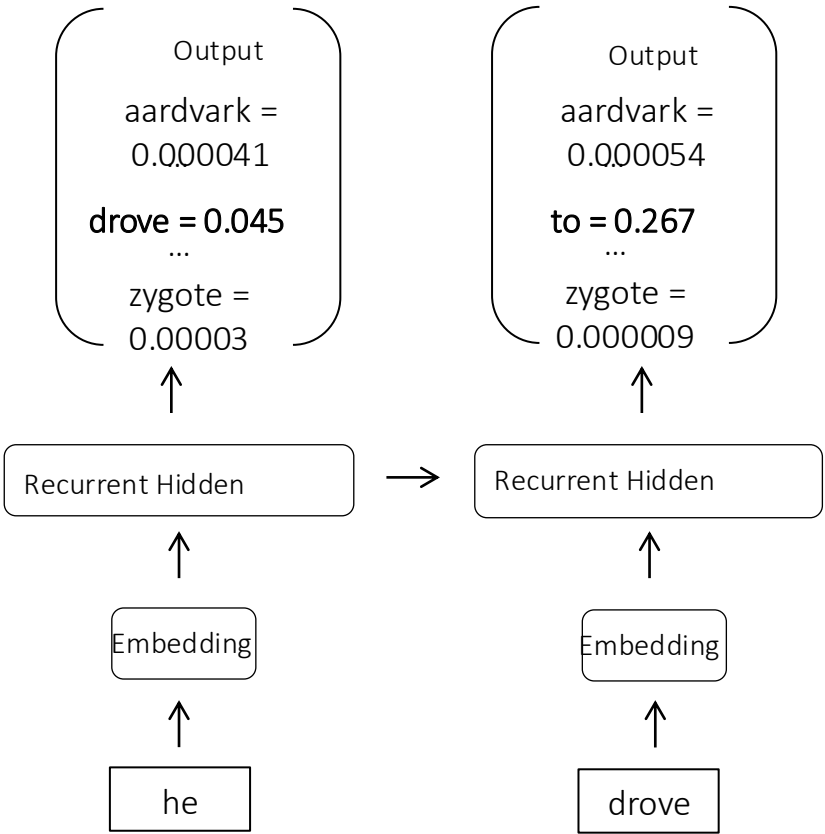


Neural Network Language Models (NNLMs)

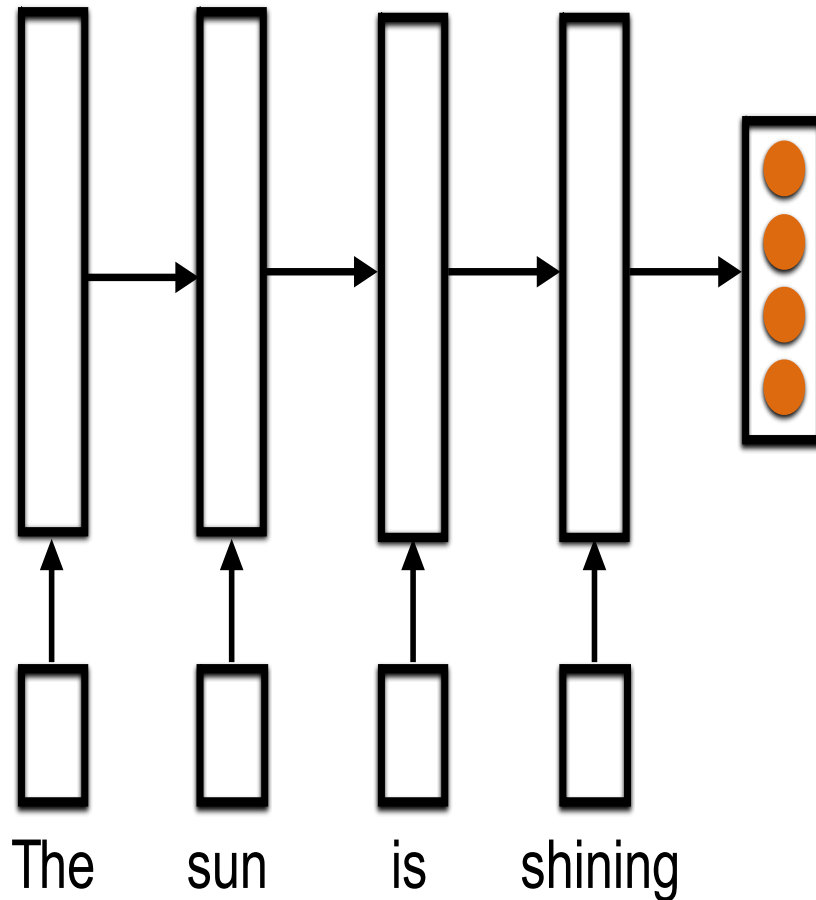
Feed-forward NNLM



Recurrent NNLM



Encoding Sentences



“Recurrent” architecture

RNN Overview

- Language Modelling
- Machine Translation

Language Modelling Problem

- Aim is to calculate the probability of a sequence (sentence) $P(X)$
- Can be decomposed into product of conditional probabilities of tokens (works):

$$P(W) = P(w(1), w(2), w(3), \dots, w(M+1)) = \prod_{k=1}^{M+1} P(w(k) | w(1), \dots, w(k-1))$$

- In practice, only finite content used

$$P(w(k) | w(1), \dots, w(k-1)) \approx P(w(k) | w(1), \dots, w(k-N+1))$$

N-Gram Language Model

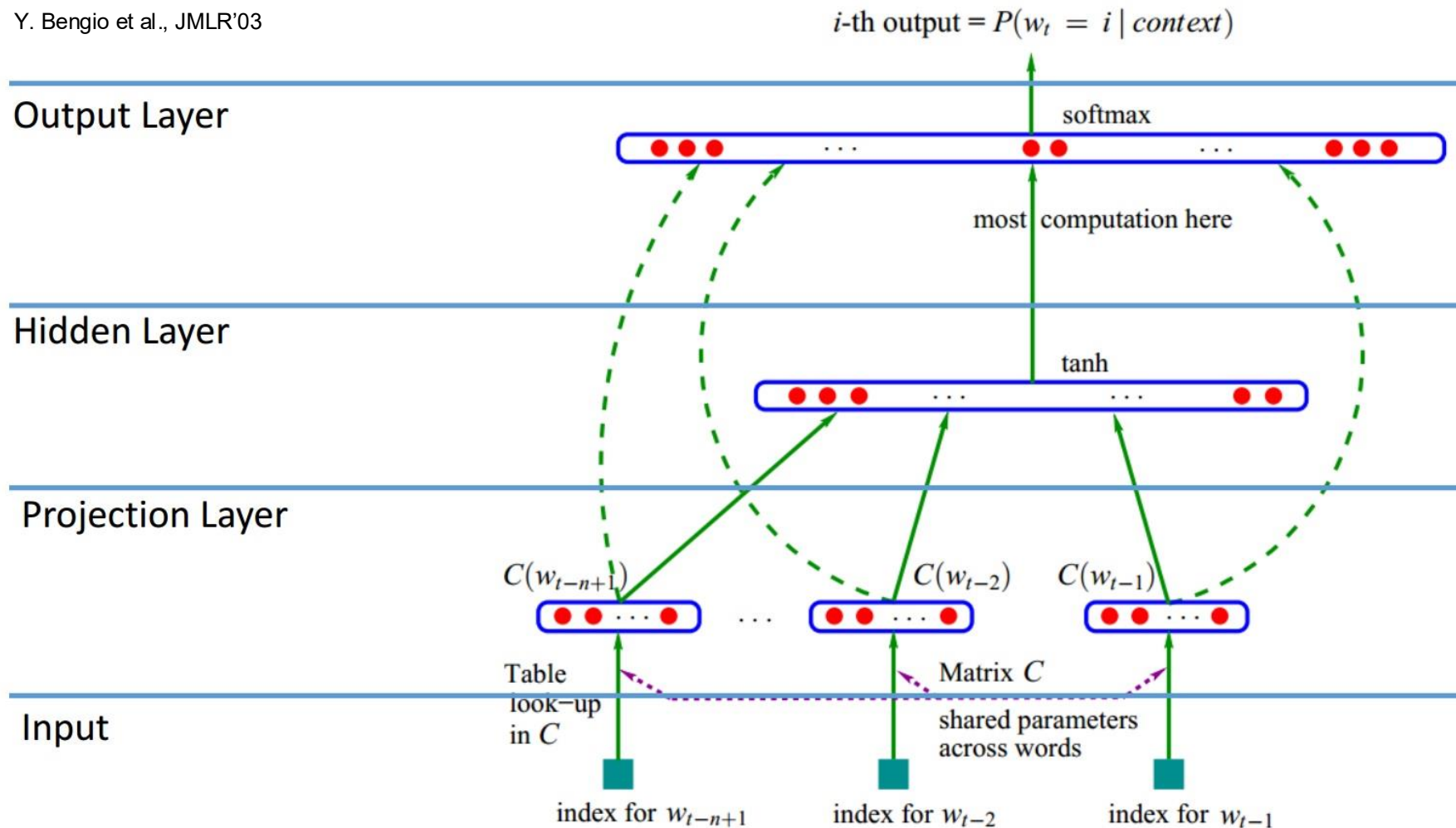
- N-Grams estimate word conditional probabilities via counting:

$$P(w(k)|h_{k-N+1}^{k-1}) = \frac{\text{Count}[w(k), h_{k-N+1}^{k-1}]}{\text{Count}[h_{k-N+1}^{k-1}]}$$

- Sparse (alleviated by back-off, but not entirely)
- Doesn't exploit word similarity
- Finite Context

Neural Network Language Model

Y. Bengio et al., JMLR'03



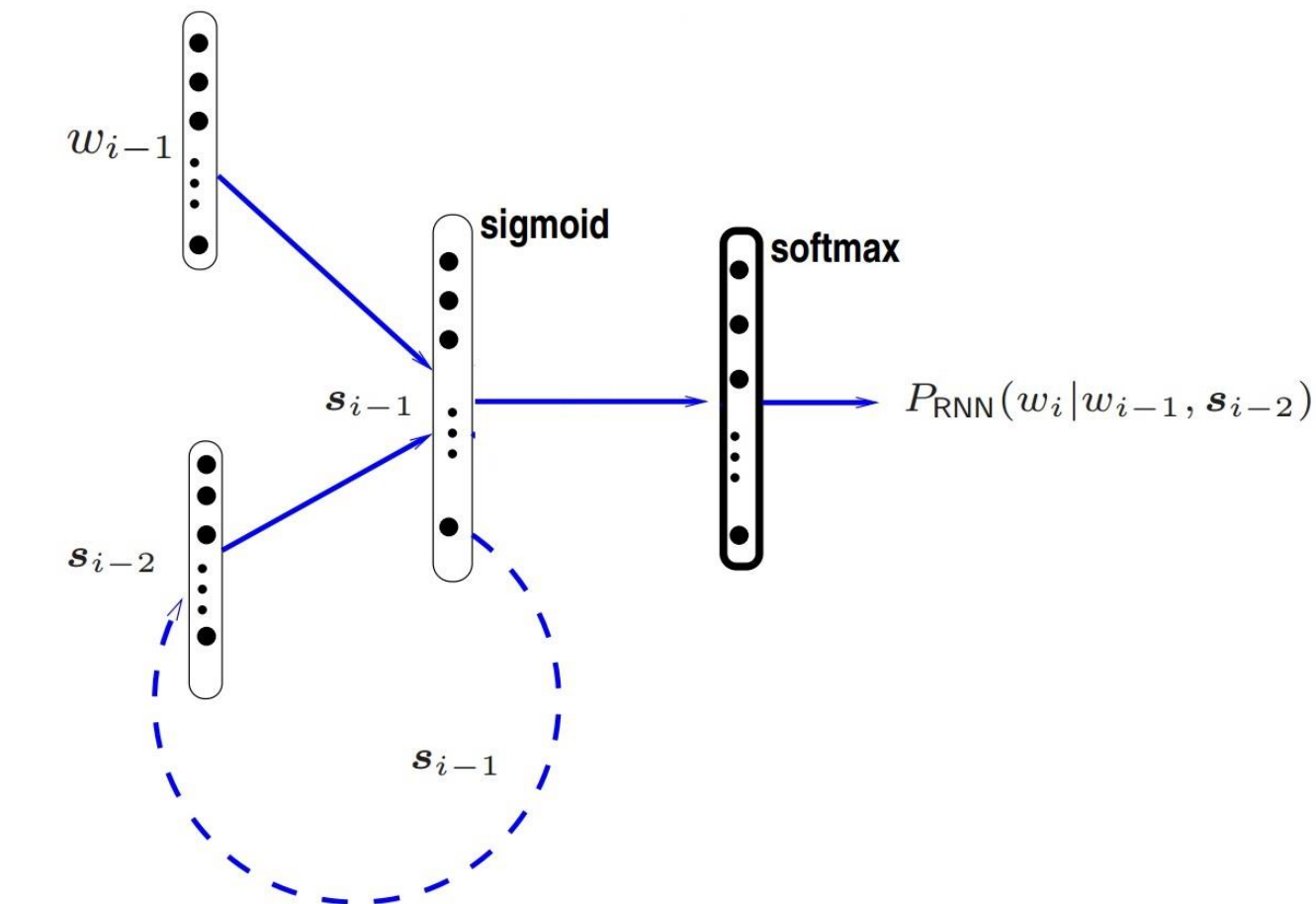
Limitation of Neural Network Language Model

- Sparsity – Solved
- World Similarity – Solved
- Finite Context – Not
- Computational Complexity - Softmax

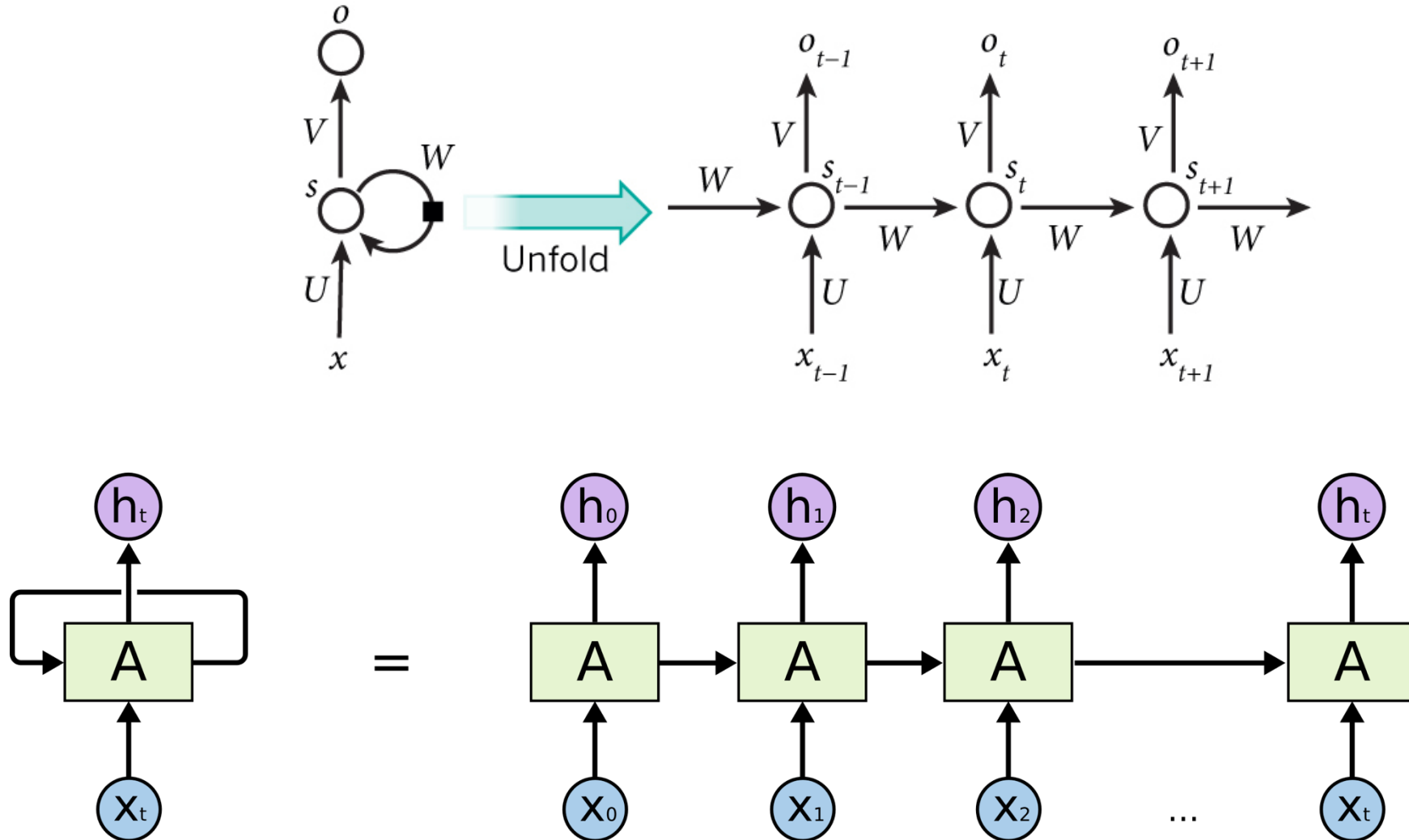
Recurrent Neural Network Language Model

[X. Liu, et al.]

Input layer Hidden layer Output layer



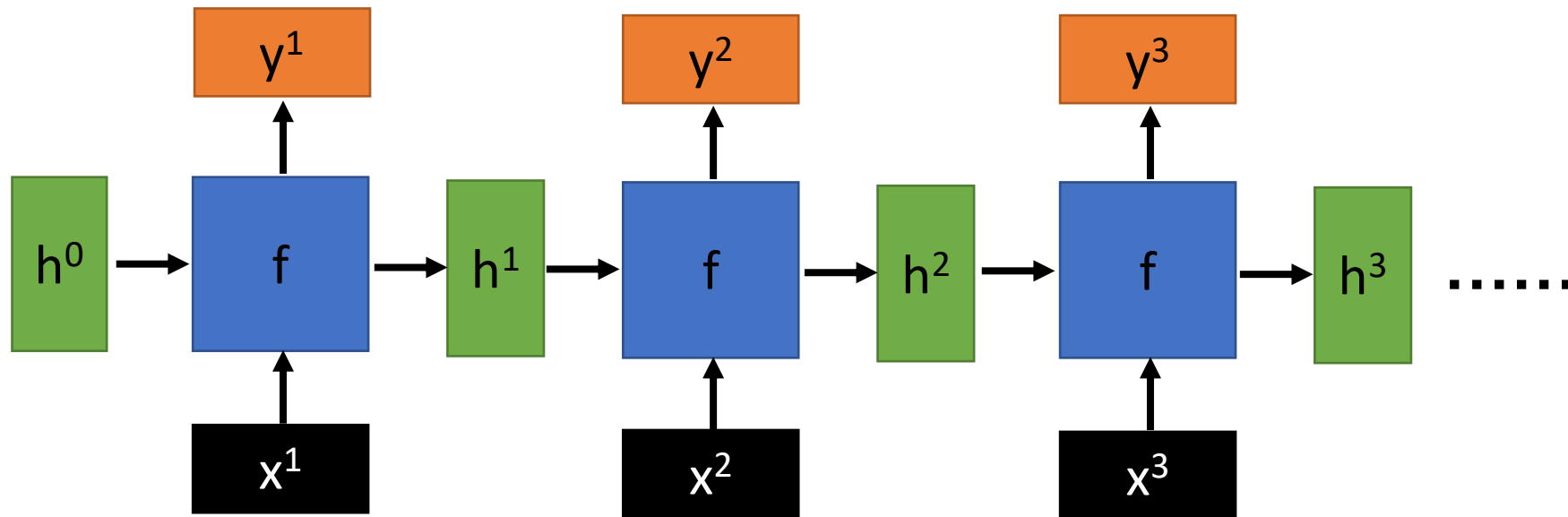
Recurrent Neural Network



How does RNN reduce complexity?

- Given function $f: h', y = f(h, x)$

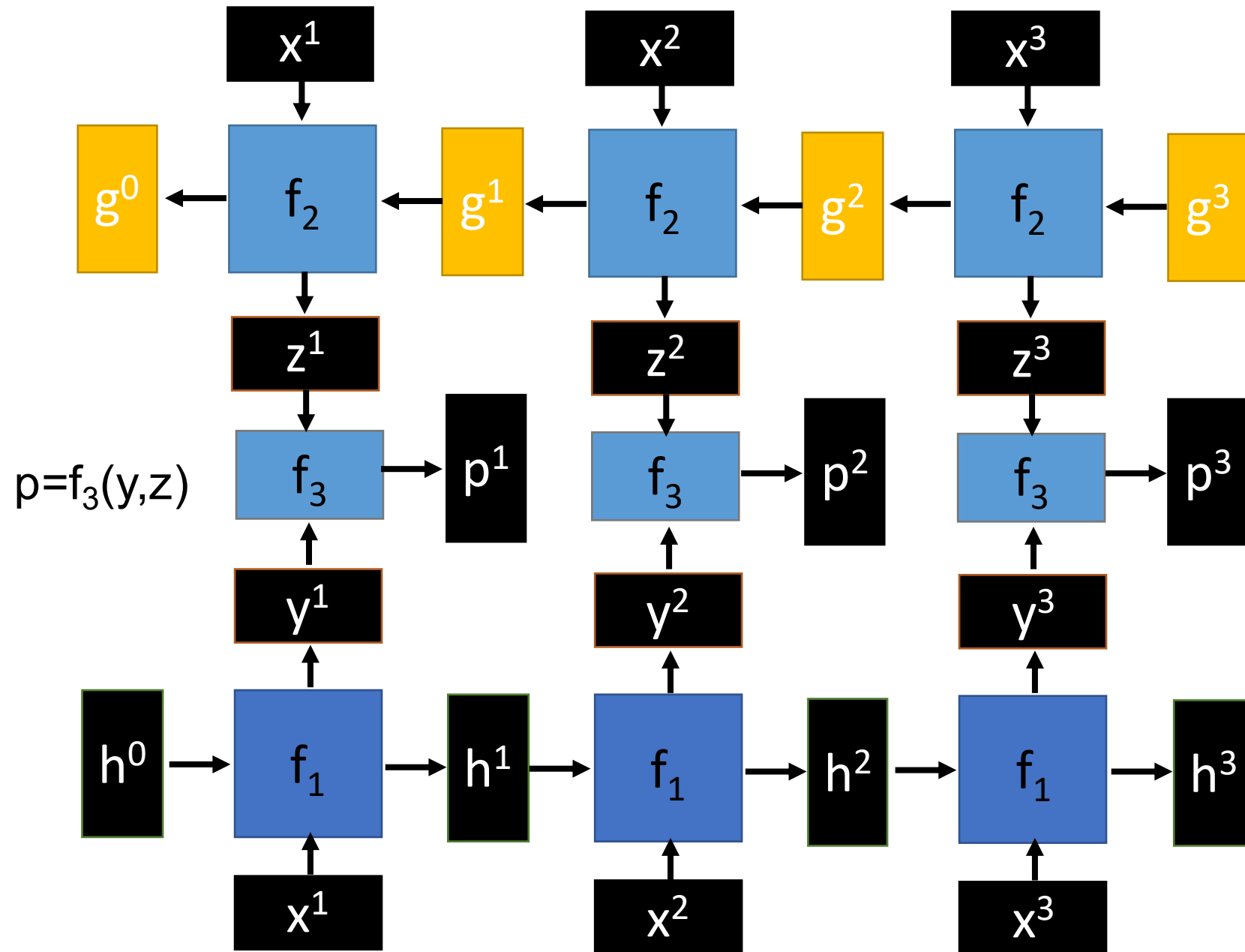
h and h' are vectors with the same dimension



No matter how long the input/output sequence is, we only need one function f . If f 's are different, then it becomes a feedforward NN. This may be treated as another compression from fully connected network.

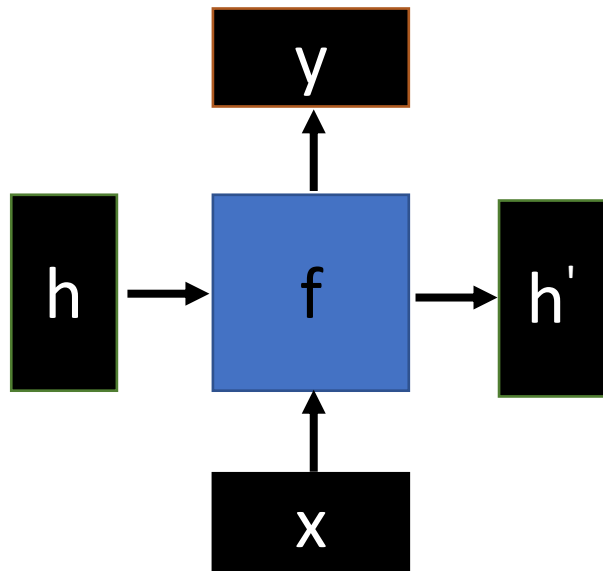
Bidirectional RNN

$$y, h = f_1(x, h) \quad z, g = f_2(g, x)$$



Naïve RNN

- Given function $f: h', y = f(h, x)$



$$h' = \sigma(W^h h + W^i x)$$

$$y = \sigma(W^o h')$$

softmax

Note, y is computed from h'

We have ignored the bias

Problems with naive RNN

- When dealing with a time series, it tends to forget old information. When there is a distant relationship of unknown length, we wish to have a “memory” to it.
- Vanishing gradient problem.

